Final Report

MAE 3780

Mechantronics

Team Robit Team (Team 44)

Gregory Sharma (gos9), Christina Ge (rg759), Kelvin Yuan (kky8)

Robot Design and Strategy Overview

Mechanical: The robot is shaped as a 7 x 7.5 x 2-inch rectangular prism where the cube enter along one entire side. The wheels were the default one provided to us while their motors were placed in the middle of the robot and a caster ball was placed in the rear opposite to the intake. To maintain weight balance and prevent the robot from tipping forward, the electronics, such as the breadboard, Arduino, and battery pack, were mounted between the main wheels and the caster wheel. The intake was made the entire length of the robot, as shown in the picture below, in order to gather as many cubes as possible on a single pass. As shown in Appendix **C.1**, there is a triangular barrier in front of the wheels for the blocks to funnel past the wheels and into the back of the robot.

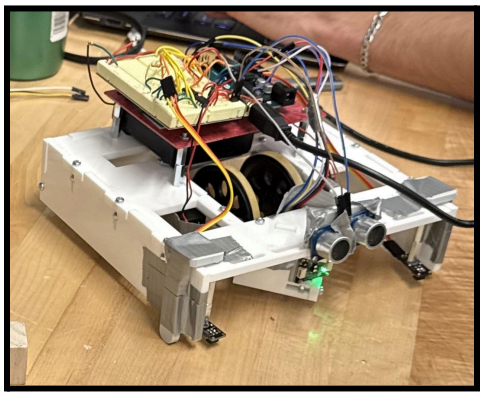
Electronics: We used two H-bridges and servos that were mounted between 2 QTI sensors that detected a black surface, which was helped by adding the 10kΩ resistor between the power ("W") and output ("R") pins. The time-of-flight sensor was mounted at cube height, and the ultrasonic sensor was just above it, all powered from the same 6V source and connected to the Arduino via different pins.

****DISCLAIMER**** One of our team's biggest oversights was relying heavily on a time-of-flight sensor, which we later discovered required a separate library to function. Avoiding the library was possible, but would have required extensive research and programming beyond our available time. As a result, the original code in Appendix **E2** was simplified to the hardcoded version in Appendix **E** to comply with competition rules. Despite this, the "before comp" code and electronics are included to showcase the work and learning our team accomplished.

*Software (**before comp**):* The code began with a startup sequence that drove the robot forward and rotated it to reach the center before the opposing robot. It then performed a 360° scan, using the time-of-flight sensor to record the closest cube at different angles. If a cube was detected within approximately 3 inches, the robot exited the search loop, drove toward the cube with a small overshoot to capture it, then rotated about 180° to secure the cube inside its body before restarting the process. If the QTI sensor triggered while driving, the robot backed up about 2 inches, turned slightly, and resumed searching. To reduce collisions, the robot also compared the cube direction with ultrasonic sensor data which measured the distance to the other robot if it was also in it's path. Our robot was programmed to avoid cubes located in the same direction as the opposing robot,

instead targeting the next closest cube. Although this approach was only an approximation because the opposing robot moved in real time, it was sufficient for our design.

*Software (**during comp**):* Looped between forward driving and right turning.



Design Process Reflection:

Initially, the robot faced significant difficulty turning since the wheels would spin so fast and slip, creating a wobble that would cause the robot to turn nonlinearly. As a result, the distance between the wheels was forced to increase to the distance shown in Appendix **C.1**. The chassis of the robot was initially meant to be 3D printed, but due to budget constraints, the manufacturing method was changed to laser cutting. This was done by dividing the CAD of the chassis into multiple pieces and creating space for the nuts and bolts to allow for proper assembly, as shown in Appendix **C.2**. This way, the chassis was manufacturable under the budget constraints. As shown in Appendix B.1, most parts were laser cut, and the budget is just under $40. Additionally, the mass of the robot is also higher with the laser cutting design, giving the robot an advantage to push off lighter robots during competition. The manufacturing process was fairly successful. The only redesign that was done was the caster wheel mount laser cut. The CAD files of the robot were not accurate, and the diameter of the cotter pin was not considered in the initial design. For the first three Milestones, the default metal chassis was used. For milestone four, the finished chassis was used, and it was successfully able to gather the required amount of blocks on the first attempt.

Competition Analysis:

Considering the disclaimer above, the robot's blind operation meant that block collection relied more on chance than on any true competitive advantage. Despite this, the block collection mechanism performed reliably, and its rapid

turning ability worked very well. However, this increased mobility came at the cost of stability and resilience in collisions with the other robot. Multiple times during the competition, our robot would headbutt the opponent and either become locked in place or be slowly pushed off the board. In addition, the rubber bands repeatedly slipping off the wheels further reduced mobility. Together, these issues significantly limited the robot's ability to remain active in the center of the board, which was critical for maximizing block collection. We found that this inconsistent traction, which the rubber bands made with the floor (especially when being tipped by the other robot), would prevent the robot from turning back to the center of the board or losing traction altogether. Additionally, the blocks would sometimes block other blocks from entering due to the narrow channels inside.

Conclusions:

For the chassis, it may have been worth exploring designs that pushed the wheels further back in order to keep the intake area more open. This would avoid bottlenecks that sometimes occurred when the cubes were traveling between the motor and the walls from the front to the back of the robot. In addition, simplifying the robot logic could have led to more efficient code and more time to test and fine-tune. That said, breaking the code into separate scripts to test each sensor individually proved extremely useful for troubleshooting both software and electrical issues. One key takeaway from this experience is that small assumptions can compound into major problems. For example, the robot's rotation angle was based on the assumption that the wheels turned at a constant rate, but slight variations in traction introduced enough inconsistency to affect positioning accuracy, which is an issue that was not fully anticipated. Similarly, the time-of-flight sensor's incompatibility with competition rules forced us to remove a major source of perception that the robot relied on for intelligent decision-making. Overall, a more thorough evaluation of both mechanical and software feasibility earlier in the design process would help our team and future teams avoid challenges that ultimately limited competitive performance.

Appendixes

Appendix A:

A.1

Bill of Materials

Item #	Name	Material/ vendor	Description	Cut(inch) /weight(g)	Unit Cost	Quantity	Total Cost
1	Acrylic sheets	acrylic	bought to cut the chassis	0	5.00	1.00	5.00
2	left and right plate	acrylic	on the final robot	23.34	1.67	1.00	1.67
3	top plate	acrylic	on the final robot	92.65	5.13	1.00	5.13
4	back plate new	acrylic	on the final robot	28.7	1.94	1.00	1.94
5	front funnel 1 new	acrylic	on the final robot	12.46	1.12	1.00	1.12
6	front funnel 2 new	acrylic	on the final robot	12.51	1.13	1.00	1.13
7	wheel v1	acrylic	boned	2.6	0.63	2.00	1.26
8	wheel v2	acrylic	on the final robot	5.42	0.77	2.00	1.54
9	distance sensor	adafruit	on the final robot	n/a	14.95	1.00	14.95
10	ultrasonic mount base	3d print	on the final robot	2.8	1.84	1.00	1.84
11	ultrasonic adjustable mount	3d print	on the final robot	2.9	1.87	1.00	1.87
12	distance sensor mount	3d print	on the final robot	2.23	1.67	1.00	1.67

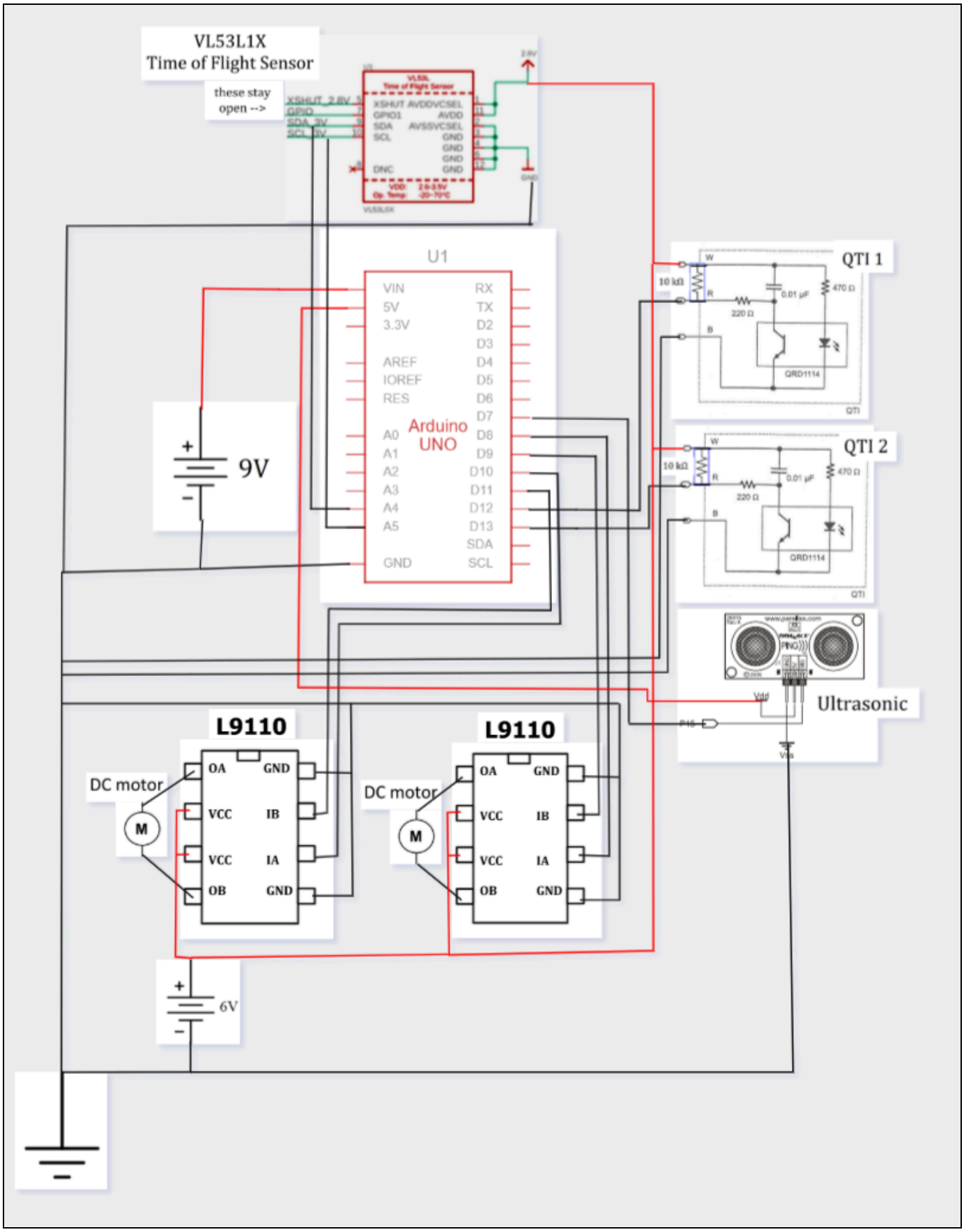
A1.2

Budget Analysis

Budget	40
Amount Used	39.11
Amount Left	0.89

Appendix B:

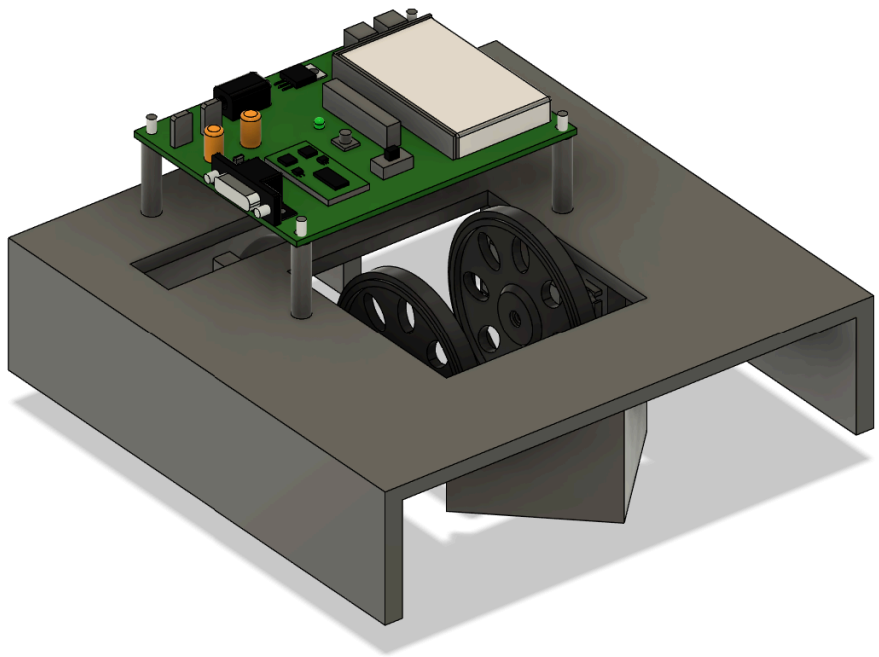
B.1 Circuit Diagram of the system (even though the sensors were plugged in during comp, they were not used)


VL53L1X
Time of Flight Sensor
these stay
open -->
XSHUT 2.8V
GPIO
SDA 3V
SCL 3V
XSHUT AVDDVCSEL
GPIO1 AVDD
SDA AVSSVCSEL
SCL GND
GND
GND
DNC GND
U1
VIN
5V
3.3V
AREF
IOREF
RES
A0
A1
A2
A3
A4
A5
GND
RX
TX
D2
D3
D4
D5
D6
D7
D8
D9
D10
D11
D12
D13
SDA
SCL
Arduino
UNO
9V
QTI 1
10 kΩ
0.01 µF
470 Ω
220 Ω
QRD1114
QTI 2
Ultrasonic
Vdd
L9110
OA
GND
VCC
IB
VCC
IA
OB
GND
DC motor
M
6V

Appendix C:

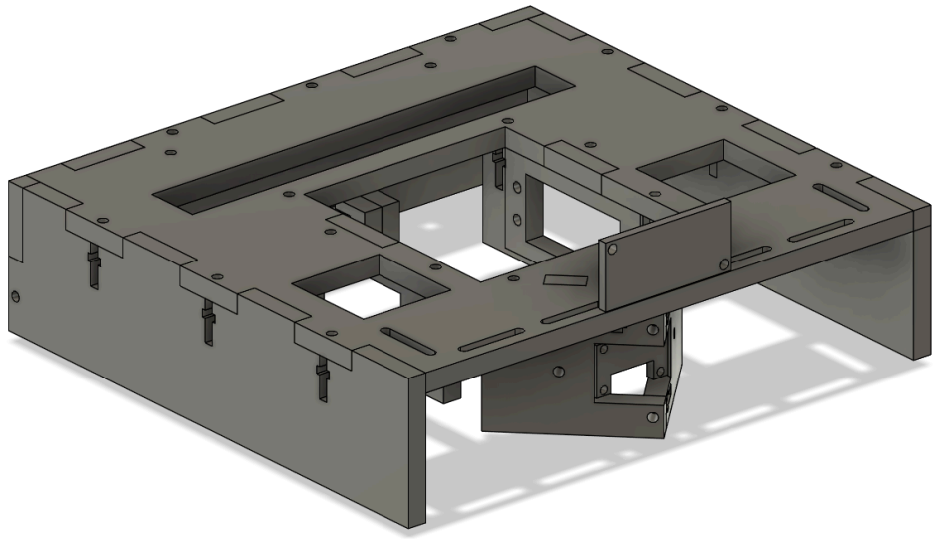
C.1

Overall CAD for Robot:



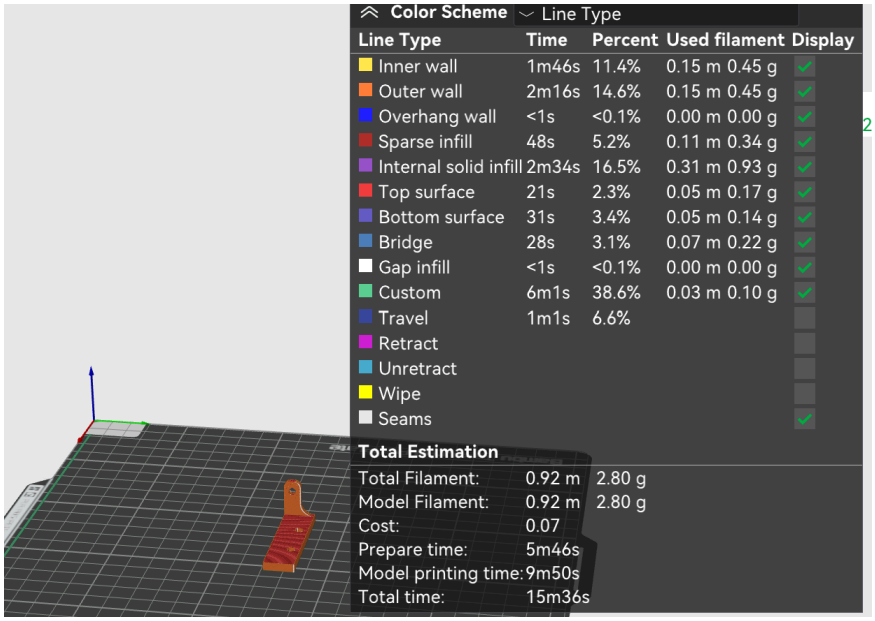
C.2

Chassis CAD for Laser Cut Manufacturing:



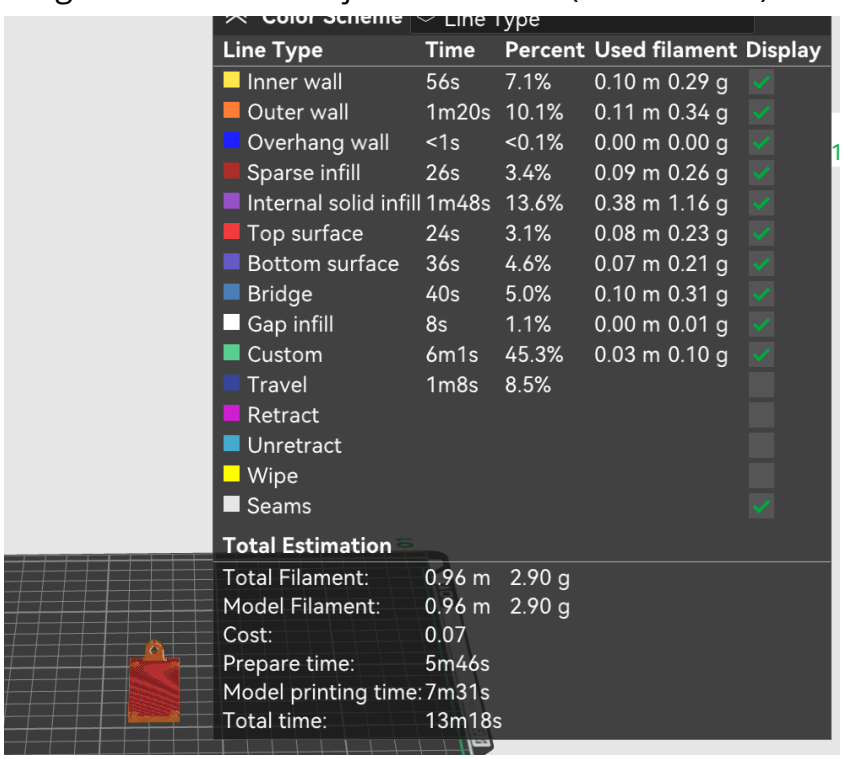
C.3

weight of ultrasonic mount base(BOM line 10)



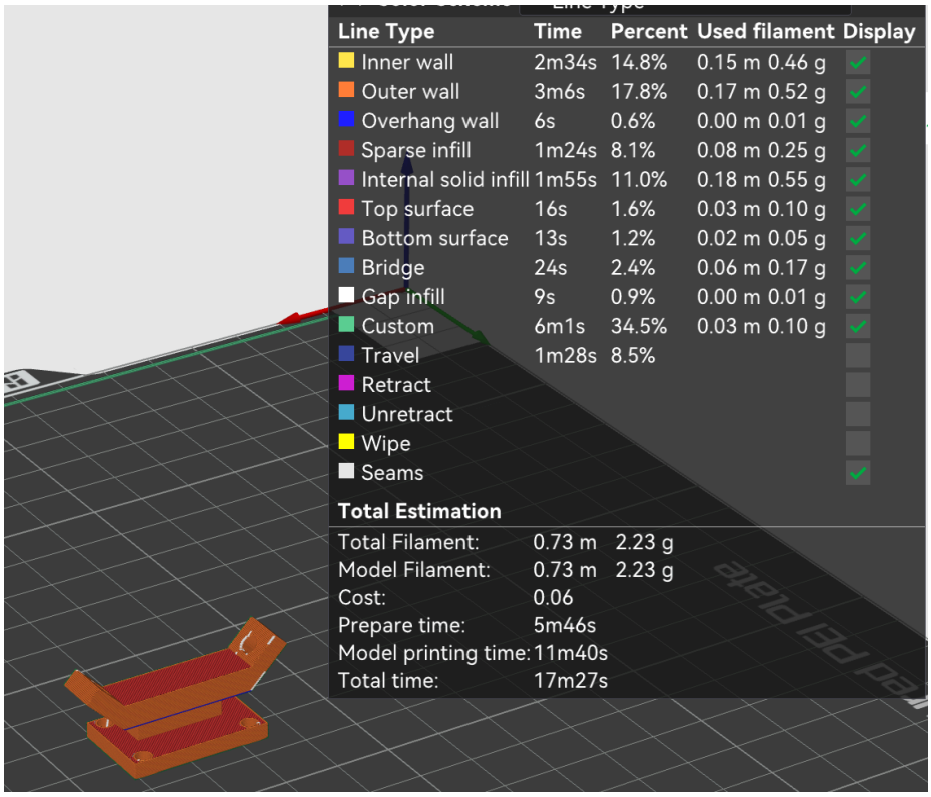
C.4

Weight of ultrasonic adjustable mount(BOM line 11)



C.5

Weight of distance sensor mount (BOM line 12)



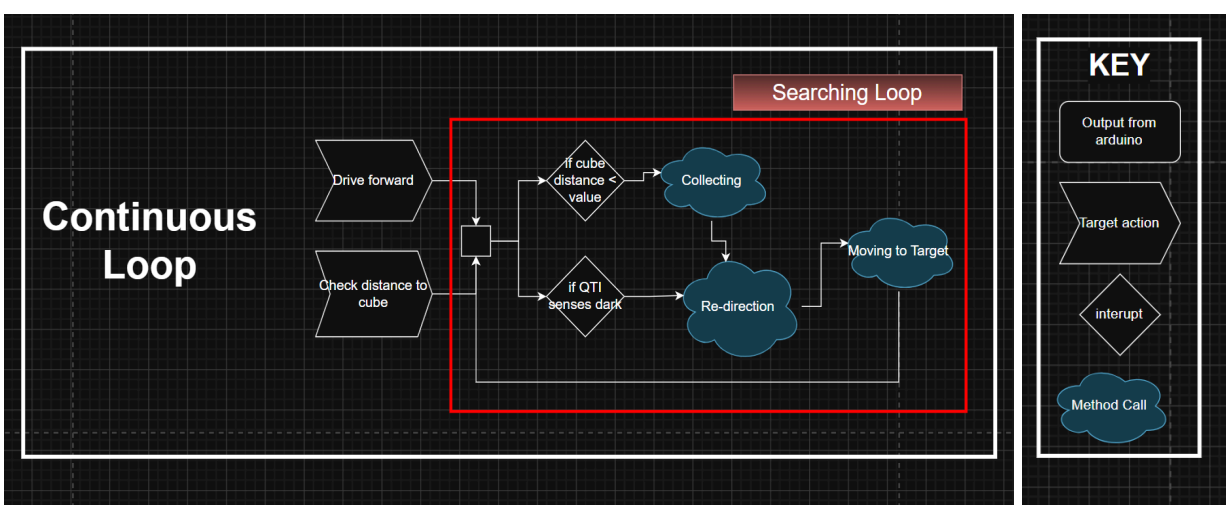
Line Type	Time	Percent	Used filament	Display
Inner wall	2m34s	14.8%	0.15 m 0.46 g	✓
Outer wall	3m6s	17.8%	0.17 m 0.52 g	✓
Overhang wall	6s	0.6%	0.00 m 0.01 g	✓
Sparse infill	1m24s	8.1%	0.08 m 0.25 g	✓
Internal solid infill	1m55s	11.0%	0.18 m 0.55 g	✓
Top surface	16s	1.6%	0.03 m 0.10 g	✓
Bottom surface	13s	1.2%	0.02 m 0.05 g	✓
Bridge	24s	2.4%	0.06 m 0.17 g	✓
Gap infill	9s	0.9%	0.00 m 0.01 g	✓
Custom	6m1s	34.5%	0.03 m 0.10 g	✓
Travel	1m28s	8.5%		
Retract				
Unretract				
Wipe				
Seams				✓

Total Estimation

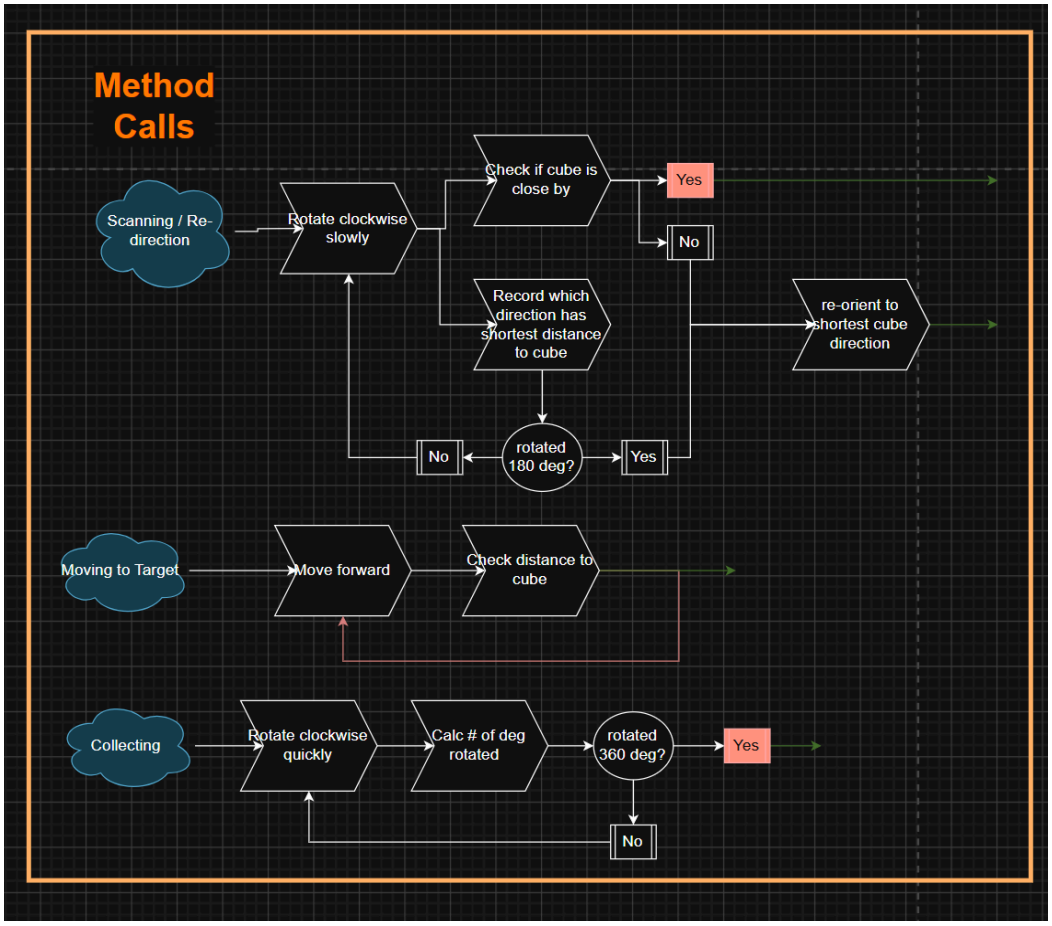
Total Filament:	0.73 m	2.23 g
Model Filament:	0.73 m	2.23 g
Cost:	0.06	
Prepare time:	5m46s	
Model printing time:	11m40s	
Total time:	17m27s	

Appendix D

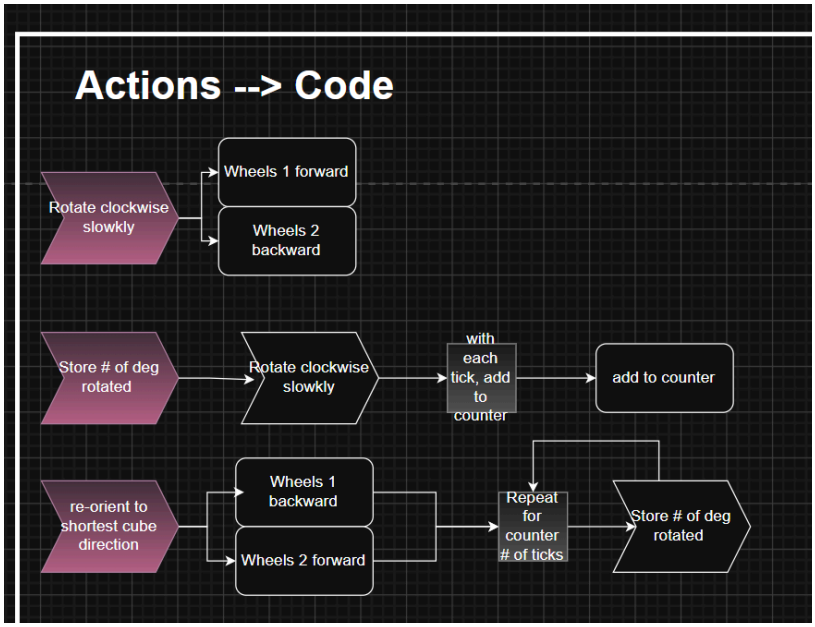
D.1

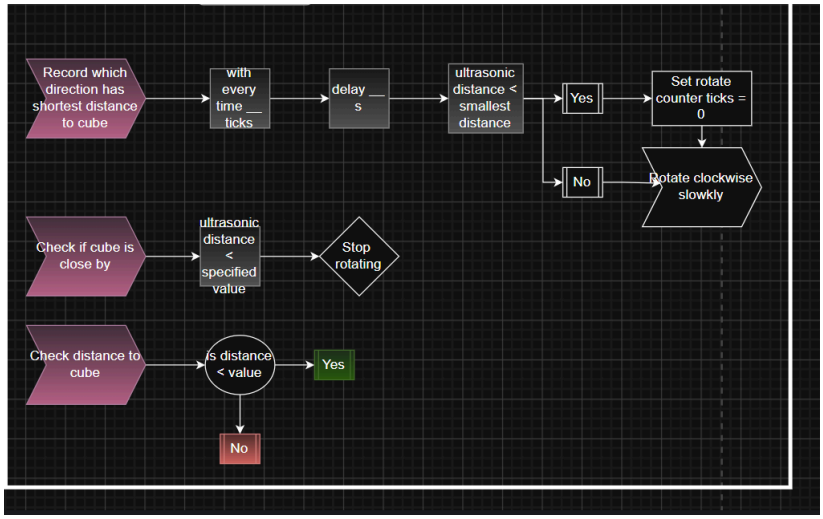


D.2


Method Calls
Scanning / Re-direction
Rotate clockwise slowly
Check if cube is close by
Yes
No
Record which direction has shortest distance to cube
re-orient to shortest cube direction
rotated 180 deg?
No
Yes
Moving to Target
Move forward
Check distance to cube
Collecting
Rotate clockwise quickly
Calc # of deg rotated
rotated 360 deg?
Yes
No

D.3





Appendix E (No library code used which was used in Competition)

```
#include <Arduino.h>

/*
+++++++++++++++ GLOBAL VARS +++++++
*/
// These are variables we change to adjust how much the robot turns and moves (essentially delay times)
double firstDist = 1.75;
double secondDist = 1;
double thirdDist = 1.25;
double dTime = 600;
// Timing scale vars
double dscale = 2500;
double tscale = 5;
// ++++++++++++++++++++++++++++

void setup() {
```

```
  Serial.begin(115200); // this is the baud rate we use now
  delay(500);
  DDRB = 0b00001111;  // motor pins
  Serial.println("Starting hardcoded motion...");
  startSequence(); //calls the method to put the robot in the center before looping
}
/*
++++++++++++++++ LOOP +++++++++++++++++++++++++++++++++
*/
void loop() {
  // run forever in a square loop path across the board
  runPattern();
  // this is much more complicated in the other fully developed code that we couldn’t use in comp.
}
/*
++++++++++++++++ MOTION LOOP +++++++++++++++
*/
void startSequence() {
  Serial.println("Start forward!");
  forwardAbit(firstDist);
  delay(dTime);
  drive_right_amount(90);
  delay(dTime);
  forwardAbit(secondDist);
  delay(dTime);
}

void runPattern() {
  while (true) {
    drive_right_amount(90);
    delay(dTime);
    forwardAbit(thirdDist);
    delay(dTime);
  }
}

/*
+++++++++++++++ MOTION METHODS +++++++++++++++++
*/

void stop() {
  PORTB &= 0b11110000;   // clear pins 8-11 that control the motors, does nothing to the others
  delay(200);
}

void drive_forward() {
  PORTB &= 0b11110000;
  PORTB |= 0b00001001; // sets 8 and 11 high (both motors forward… you get the idea for the right left and back methods)
}
void drive_back() {
  PORTB &= 0b11110000;
  PORTB |= 0b00000110;
}
void drive_right() {
  PORTB &= 0b11110000;
  PORTB |= 0b00000101;
}
void drive_left() {
  PORTB &= 0b11110000;
  PORTB |= 0b00001010;
}
/*
+++++++++++++++ prescribed distances methods +++++++++++++++++++++
*/
void forwardAbit(double dist) {
  drive_forward();
  delay(dist * dscale);
  stop();
}
```

```
void drive_right_amount(double ang) {
  drive_right();
  delay(ang * tscale);
  stop();
}
void drive_left_amount(double ang) {
  drive_left();
  delay(ang * tscale);
  stop();
}
```

Appendix E2 (Fully Sensor Integrated code with the necessary library used for Time of Flight sensor). Not completely calibrated but the sequence and setup is correct.

```
// Team Robit Team
/*
+++++++++++++++++ GLOBALS+++++++++++++++++
*/
#include <Arduino.h>
#include "Adafruit_VL53L1X.h"  // Library needed to get data from time of flight sensor THAT IS IT
Adafruit_VL53L1X vl53 = Adafruit_VL53L1X(-1, -1);

// Tuning Variables with each test
double dscale = 2500;                       // this is multiplied by distance in INCEHS to get delay
double tscale = 4.4;                          // raise to increase the amount it should turn left WHEN ADJUSTING TO CUBE
double threshold = 1;                     // distance cube needs to be before it auto-goes for it and stops searching for others
double block_to_other_robot_distance = 2;  // if the robot is detected to be _____ far away from the block, then it can go for
that block. this is to avoid having the robot go for blocks that are obstructed by the other robot
double overshoot_dist = 2;                  // This is the amount the robot should overshoot to get the block
double ultraSonic_speed_scale = 148.0;     // this value is being divided by the time durration and converts to inches.
double minRobotDist = 1;                  // if the other robot is beyond this distance, it is considered to  "not be in the way"
double time_angScale = .00028;    // this is WHEN SCANNING (increase to increase angle)
double calc_dist_time_distScale = 10000;
double time_distScale = 155000;  // this is what is multiplied by time to get distance WHEN CHASING.
// Startup Variables
int numCubes = 16;
volatile double Delaytime = 1000;
// Recording Variables
volatile long tstart = 0;
volatile long tend = 0;
volatile double deltaT = 0;
volatile bool isRobotInWay = false;
volatile bool QTI_trig = false;
volatile double current_angle = 0;    // this should be a rough guess which direction it is pointing in based on every other time
it has rotated
volatile double adjust_ang_left = 0;  // counterclockwise angle adjustment
volatile double dist_to_cube = 10;
volatile double dist_to_otherbot = 100000; //initial guess (that there is no robot in the way)
// Reference Variables
volatile bool reset_lastB = false;
double startCol = 0;
double backupD = 0.1;
volatile uint16_t period = 0;  //  stores the value of the output wave period (not necessarily in seconds (this wworks
/*
+++++++++++++++ INTERUPTS +++++++++++++++
*/
// INTERUPT: Any of the QTI's detect black
ISR(PCINT0_vect) {
  static uint8_t lastState = 0;
  static unsigned long lastTrig = 0;
  uint8_t current = PINB;
  uint8_t rising = (~lastState) & current;  // this is so it only triggers on a rising edge (so it doesn't like jitter-trigger)
  unsigned long now = micros();
  if ((rising & 0b00110000) && (now - lastTrig > 50000)) {  // this is to make sure it doesn't trigger so many times in one
"instant"
  QTI_trig = true;
  lastTrig = now;
}
```

```
  lastState = current;
}
/*
++++++++++++++++ METHODS TO CALL! ++++++++++++++++
*/
double timeElapsed(long t1, long t2) {
  return (double)(t2 - t1); // just do the calculation outside to neaten things
}

double calcAng(double t) {
  return t * time_angScale; // same here
}

double calcDist(double d) {
  return d * calc_dist_time_distScale;  // and here
}

double bottomDist_laser() {  // returns reading from laser
  // Wait until data is ready. This is the setup we got from the code given to us from the datasheet (it is to detect the sensor
and if this doesn't work, the robot will never start which is why we have other testing scripts outside of this code.
  while (!vl53.dataReady()) {
  }

  int16_t distance = vl53.distance();
//  Serial.print("laser value: ");  old print statements i would use when debugging (i will leave them here)
//  Serial.println(distance);
  if (distance == -1) {
    Serial.print(F("Error reading distance: "));  // code is not ready to handle if this is -1 so fix this as soon as it happens
if it does
    Serial.println(vl53.vl_status);
    return -1;
  }
  // Clear interrupt so next reading can happen (also code from the datasheet we got)
  vl53.clearInterrupt();
  return distance * 0.03937008;  // THIS CONVERTS FROM mm TO INCHES!!
}

void getRobotDist() {      // this is essentially where we read the echo back from the pulse to determine the distance away
  unsigned long startTime, duration;  // used to track
  // trigger the call
  DDRD |= 0b10000000;    // OUTPUT: D7
  PORTD &= ~0b10000000;  // LOW:    D7
  delayMicroseconds(2);
  PORTD &= ~0b10000000;  // LOW:    D7
  delayMicroseconds(5);
  // now listen for echo
  DDRD &= ~0b10000000;   // INPUT:  D7
  PORTD &= ~0b10000000;  // pull-up disabled: D7
  unsigned long timeoutStart = micros();
  // Wait for pulse to go HIGH
  while ((PIND & 0b10000000) == 0) {
    if (micros() - timeoutStart > 30000) {  // wait 0.03s, if nothing, then say there is no robot
      isRobotInWay = false;
      dist_to_otherbot = 1000000;  // treat as "no robot detected"
      return;
    }
  }
  startTime = micros();
  while (PIND & 0b10000000) {  // this checks if the ultrasonic stays high for too long, then stop and say there was a no robot
in the way
    if (micros() - startTime > 30000) {
      isRobotInWay = false;
      dist_to_otherbot = 1000000;  // treat as no robot detected
      return;
    }
  }
  duration = micros() - startTime;
  double distanceMesured = (duration / ultraSonic_speed_scale);
  if (distanceMesured > minRobotDist) {  // this is used to check if the robot is actually in the way or not within the margin of
error we are worried about (if might be in line but too far for us to actually care about)
```

```
    isRobotInWay = false;
    dist_to_otherbot = 1000000;  // returns a really large number which will always be greater than the distance to a cube
  } else {
    isRobotInWay = true;
    dist_to_otherbot = distanceMesured;  // change the global var
  }
}

double scanning() {  // this method turns the robot right and scan for the closest block. returns the angle needed to turn LEFT
to face closest block that isn't blocked by robot.

// the way I chose to rotate and keep track of the robot angle in this case is inconsistent since the robot does not rotate at a
constant rate (which is due to mechanical errors). This is a big problem but with some tweaking, can minimize the issue.

  Serial.println("Scanning!");
  bool keepScan = true;
  double deg_turned = 0;
  double target_block_angle = 0;
  double closest_cube_dist = 10000;  // make sure this isn't an issue
  drive_right();
  long t_temp_end = micros();
  while (deg_turned < 360 && keepScan == true) {  // MAYBE have make a decisions at angles less and 360 !!!!!!!!!!! figure out
with testing
    t_temp_end = micros();
    deg_turned = calcAng(timeElapsed(tstart, t_temp_end));
    getRobotDist();
    double cube_dist = bottomDist_laser();
//      Serial.print("BUFFER DIST: ");
//      Serial.println(dist_to_otherbot - block_to_other_robot_distance);
    if (isRobotInWay == false) {
      if (cube_dist < threshold) { // This is if the robot sees a cube that is REALLY close by (threashold) and where the robot
is not in the way. Then it is programmed to go for it immediately.
        Serial.print("LESS THAN THRESH: ");
        keepScan = false; // breaks the loop and enters the next stage (getting the block)
        target_block_angle = deg_turned;
      } else if (cube_dist < closest_cube_dist) { // if it sees a block that is closer than the closest block (but isn't REALLY
close) it marks that angle as the angle it should return to in order to get this NEW closest block.
        closest_cube_dist = cube_dist;
        target_block_angle = deg_turned;
      }
    }
  }
  if (closest_cube_dist == 10000) {  // if it didn't find a block (this is usually an error)
    Serial.println("NO BLOCK FOUND!");
    stop();
    return 180;  // have it just turn around 180 deg LEFT and try somewhere else
  }
      Serial.print("CUBE FOUND: ");
          Serial.println(closest_cube_dist);
  stop();
  return (deg_turned - target_block_angle);  // return the angle moved - the angle needed to be at (this is the amount of angle
needed to turn left)
  // rotated and scan
  // if the block distance < threshold, go for that block
  // else, go for the block with the shortest distance
  // if no blocks found and on the color it didnt start on, point in 180 deg direction
  // else point in the 0 deg direction
}

void adjusting(double angleLeft) {  // move left by this amount in order to get back to the angle that the closest cube is
sitting at
  Serial.print("Adjusting! by: ");
  Serial.print(angleLeft);
  Serial.println(" deg");
  drive_left_amount(angleLeft);
}

void chasing() {
  // moves forward the distance needed to get there
  bool stopChase = false;
```

```
  long t_temp_end = micros();
  double deltT = 0;
  Serial.println("Chasing!");
  drive_forward();
  while (stopChase == false && QTI_trig == false) {  // if the QTI is triggered, stop chasing and skip to next step. this is to
avoid being outside:: THIS WOULD BE A PROBLEM IF THERE IS A CUBE OUT OF BOUNDS THAT IT IS TRYING TO GET!!!
    t_temp_end = micros();
    deltT = timeElapsed(tstart, t_temp_end);
    Serial.print("DIST TRAVELED:  ");
    Serial.println(deltT / time_distScale);
    if ((deltT / time_distScale) > (dist_to_cube + overshoot_dist)) { // needs to move forward after the total delay has passed
(both the distance needed to get to the cube, and the distance more it needs to go to swallow it).
      stopChase = true;
    }
  }
  if (QTI_trig == true) {  // get rid of this condition if the QTI is bugging
    Serial.println("QTI triggered: alternate plan: ");
    stop();
    backupAbit(backupD);  // backup before scanning again
    QTI_trig = false;
  }
  stop();
  Serial.println("Done Chasing!");
}

void securing() {
    Serial.println("SECURING!");
  // just do a little spin and record how much it spun to log the block inside the mechanism under the robot.
  drive_left_amount(180);
    Serial.println("DONE SECURING!");
    Serial.println(" ");
}
/*
+++++++++++++++++++++++++++++++ SETUP +++++++++++++++++++++++++
*/
void setup() {
  Serial.begin(115200);
  delay(500);
  DDRB = 0b00001111;    // pins 8-11 as outputs (for the motors)
  DDRD &= 0b11111011;   // pin D2 set as INPUT (for the color sensor) (clears PD2)
  PORTB &= 0b11111011;  // pin D10 set LOW (clears PB2 output)
  PCICR |= 0b00000100;  // this is PORTD  (first bit is for pins D8-D13, second is for A0-A5, third is for D0-D7.)
  PCMSK2 = 0b00000000;  // this determines which pins can trigger the interupt: in this case doesnt matter
  // for QTI sensor:
  PCICR |= 0b00000001;
  PCMSK0 = 0b00110000;  // D12 + D13

  // Setup the time of flight sensor (TOF sensor)
 //PLEASE NOTE THIS WAS CODE GIVEN TO USE FROM THE DATASHEET IN WHICH WE GOT THE TOF SENSOR!!!
  while (!Serial) delay(10);
  Wire.begin();
  if (!vl53.begin(0x29, &Wire)) {
    Serial.print(F("Init failed: "));
    Serial.println(vl53.vl_status);
    while (1);
  }
  Serial.println(F("Sensor OK"));
  delay(200);  // allow to settle, this helps some of the bugging
  if (!vl53.startRanging()) {
    Serial.print(F("Couldn't start ranging: "));
    Serial.println(vl53.vl_status);
    while (1);
  }
  vl53.setTimingBudget(50);
  // Done with TOF sensor
  sei(); // enable interupts
  Serial.println("Starting ...");
  Serial.println("");
  startSeq();
}
```

```
/*
++++++++++++++++ FUNCTION LOOP +++++++++++++++++++++++++++
*/
void startSeq() {  // this is the code that the robot does once at the start every time before entering search loop
                   // should turn a bit diagonal, go forward a set amount, and wipe the board
                   // NEVER FINISHED didn’t have time
  Serial.println("start sequence does nothing rn");
  forwardAbit(0.9);
}

void loop() {
  while (numCubes > 0) {
    adjust_ang_left = scanning();  // find the angle to the closest block by turning left after it overshot (or not)
    delay(100);
    adjusting(adjust_ang_left);    // move to the right angle
        delay(100);
    chasing();                     // get the block
        delay(100);
    securing();                    // secure it by turngin a bit
        delay(200);
    numCubes = numCubes - 1;
  }
  Serial.println("Finished!");
  delay(1000000);
}

/*
+++++++++++++++ MOTION METHODs! +++++++++++++++++++++
*/
void stop() {
  Serial.print("STOP: ");
  PORTB &= 0b11110000;  // clear motor pins
  tend = micros(); // sets the time after the stop command has been executed (used to calculate dt)
  deltaT = timeElapsed(tstart, tend);  // calculates the time past as soon as the robot stops
  delay(Delaytime);                    // a pause to see actions more clearly
}

void drive_forward() {
  Serial.print("forward: ");
  PORTB &= 0b11110000;
  PORTB |= 0b00001001;
  tstart = micros(); // a variable constantly read to measure how much time has pasted to determine how far forward the robot has
moved (or with the other methods, how far back, right, left it has moved).
}

void drive_back() {
  PORTB &= 0b11110000;
  PORTB |= 0b00000110;
  tstart = micros();
}

void drive_right() {
  Serial.print("Turning right...: ");
  PCMSK0 = 0b00000000;  // disable QTI during backup just so it goes back for sure!!
  PORTB &= 0b11110000;
  PORTB |= 0b00000101;
  tstart = micros();  // start timer
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
}

void drive_right_amount(double ang) {
  Serial.print("Turning right an amount: ");
  PCMSK0 = 0b00000000;  // disable QTI during backup just so it goes back for sure
  PORTB &= 0b11110000;
  PORTB |= 0b00000101;
  Serial.print("TURNING AROUND amount: ");
  delay(ang * tscale);
  stop();               // make sure this isn't changing tend in a bad way that isn't overriden next time.
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
}
```

```
void drive_left() {
  Serial.print("Turning left:... ");
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
  PORTB &= 0b11110000;
  PORTB |= 0b00001010;
  tstart = micros();  // start timer
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
}

void drive_left_amount(double ang) {
  Serial.print("Turning left an amount: ");
  PCMSK0 = 0b00000000;  // disable QTI during backup just so it goes back for sure
  PORTB &= 0b11110000;
  PORTB |= 0b00001010;
  Serial.print("Turnin LEFT amount: ");
  delay(ang * tscale);
  stop();
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
}

void backupAbit(double dist) {
  Serial.print("Backing up: ");
  PCMSK0 = 0b00000000;  // disable QTI during backup just so it goes back for sure
  PORTB &= 0b11110000;
  PORTB |= 0b00000110;
  Serial.println("backin up");
  delay(dist * dscale);
  stop();
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
  reset_lastB = true;
}

void forwardAbit(double dist) {
  Serial.print("FORWARD MARECH: ");
  PCMSK0 = 0b00000000;  // disable QTI during backup just so it goes back for sure
  PORTB &= 0b11110000;
  PORTB |= 0b00001001;
  delay(dist * dscale);
  stop();
  PCMSK0 = 0b00110000;  // re-enable QTI after backup
  reset_lastB = true;
}
```